

Appunti di Core Java 21

Emanuele Romano

Indice

1	Fondamenta	1
1.1	Editing, Compilazione ed Esecuzione	1
1.2	Cenni alla JShell: L'approccio REPL	1
1.3	Tipi Primitivi	2
1.3.1	Il tipo char e Unicode	2
1.3.2	Il tipo boolean	3
1.3.3	Quadro generale dei tipi primitivi	3
1.4	Stringhe	4
1.4.1	Eccezioni allo String Pool: Stringhe Dinamiche	5
1.4.2	L'Immutabilità in Java: Oltre le Stringhe	5
1.5	La Classe StringBuilder	6
1.6	Input e output su console	7
1.6.1	Scanner	7
1.6.2	Output su Console	7
1.7	Lettura e Scrittura su File	8
1.7.1	Lettura da File	8
1.7.2	Scrittura su File	8
1.7.3	Gestione dei Percorsi e Eccezioni	9
1.8	Il Costrutto Switch	9
1.8.1	Lo Switch Tradizionale (Statement)	9
1.8.2	La Sintassi a Freccia (Modern Switch Statement)	9
1.8.3	Switch Expressions	10
1.8.4	Evoluzioni Avanzate: Pattern Matching e Null	10
1.8.5	L'Utilizzo della Clausola <code>when</code> negli <code>switch</code>	11
1.9	Big Numbers: BigInteger e BigDecimal	11
2	Classi e oggetti	13
2.1	I Costruttori	13
2.1.1	Ulteriori note su <code>this</code>	14
2.2	Overloading e Firma dei Metodi	14
2.3	Costruzione degli Oggetti	15
2.3.1	Inizializzazione e Valori di Default	16
2.3.2	Blocchi di Inizializzazione	16
2.3.3	L'Ordine di Esecuzione	16
2.4	L'inferenza del tipo: la parola chiave <code>var</code>	16
2.5	Ulteriori note sull'incapsulamento	17
2.5.1	Il rischio della fuga di riferimenti (<i>Leaking References</i>)	17
2.6	I Metodi Privati	18
2.7	Campi di istanza <code>final</code>	19
2.8	Membri Statici: Campi e Costanti	19
2.8.1	Campi Statici (Variabili di Classe)	19
2.8.2	I Metodi Statici	20
2.9	Il Metodo <code>main</code>	20

2.9.1	Novità Java 21: Instance Main Methods (Preview)	21
2.10	Il passaggio dei parametri: Call-by-Value	21
2.11	Metodi con numero variabile di argomenti (Varargs)	22
2.12	Tipi Enumerati (Enum)	23
2.13	I Records: classi come contenitori di dati	25
2.14	Gestione della Memoria: Stack vs Heap	27
2.14.1	Confronto Sintetico	27
3	Aspetti di Gestione del Codice	29
3.1	Anatomia di un Progetto Java: Convenzioni e Struttura	29
3.2	I Package e il Naming Convention	29
3.2.1	Collaborazione e Conflitti	30
3.3	Creazione e Gestione Manuale dei Package	31
3.4	L'Accesso a livello di Package	32
3.4.1	Sicurezza e Moduli	33
3.5	Gli Import	33
3.5.1	Risoluzione dei Conflitti di Omonimia	34
3.5.2	Gli Import Statici	34
4	Ereditarietà e Polimorfismo	36
4.1	Classi, Superclassi e Sottoclassi	36
4.2	Sovrascrivere i Metodi (Overriding)	37
4.2.1	L'importanza di <code>@Override</code>	38
4.3	I Costruttori delle Sottoclassi	39
4.4	Polimorfismo	40
4.4.1	Il Legame Dinamico (Dynamic Binding)	41
4.4.2	Sintesi Mnemonica: Il Senso di Marcia	42
4.4.3	Covarianza dei Tipi di Ritorno	42
4.5	Prevenire l'Ereditarietà: Il modificatore <code>final</code>	43
4.6	Il Casting degli Oggetti	45
4.7	Evoluzione di <code>instanceof</code> : Pattern Matching	45
4.7.1	Scoping e Operatori Logici	46
4.7.2	Il Record Pattern e la Decostruzione (Java 21+)	47
4.8	Accesso Protetto: Il modificatore <code>protected</code>	48
4.9	La Classe <code>Object</code>	49
4.9.1	Il Metodo <code>equals()</code>	49
4.9.2	Il Metodo <code>hashCode()</code>	53
4.9.3	Il Metodo <code>toString()</code>	55
4.9.4	Il caso particolare dei Records	55
4.10	Classi Astratte	56
4.11	Classi Sigillate (<i>Sealed Classes</i>)	57
5	Interfacce, Lambdas e Inner Classes	58
5.1	Interfacce in Java	58
5.1.1	L'interfaccia <code>Comparable</code>	59
5.1.2	Interfacce vs Classi Astratte	62
5.1.3	Metodi Default: Evoluzione e Compatibilità	63
5.2	Il Pattern Callback: Passare Comportamento	63
5.2.1	Un esempio di Callback: l'interfaccia <code>Comparator</code>	64

5.3	Lambda Expressions	65
5.3.1	Introduzione e Motivazioni	65
5.3.2	Sintassi delle Lambda Expressions	66
5.3.3	Uso corretto delle Lambda Expressions	67
5.3.4	Interfacce Funzionali Standard	69
5.3.5	Method e Constructor References	70
5.3.6	Accesso alle variabili (Variable Capture)	71
5.4	Inner Classes	72
5.4.1	Struttura e Accesso allo Stato	72
5.4.2	Local Inner Classes	74
5.4.3	Meccanica della Variable Capture	75
5.4.4	Classi Anonime	76
5.4.5	Classi Statiche	76
5.4.6	Il Pericolo del Memory Leak nelle Inner Classes	77
5.5	Approfondimento: le Chiusure (Closures)	78
6	Eccezioni, Asserzioni e Logging	80
6.1	Gestione delle Eccezioni	80
6.1.1	La Gerarchia delle Eccezioni	82
6.1.2	Dichiarare e Lanciare Eccezioni	83
6.1.3	Il Blocco <code>try-catch</code> : gestione locale	83
6.1.4	La clausola <code>throws</code> : gestione delegata	85
6.1.5	Ereditarietà ed Eccezioni	86
6.1.6	Creare Classi di Eccezione Personalizzate	87
6.1.7	Exceptions Chaining	87
6.1.8	<code>try-with-resources</code>	88
6.1.9	Costruttori e Metodi di Eccezioni	91
6.2	Le Asserzioni (Assertions)	91
6.2.1	Attivare/Disattivare Asserzioni	92
6.2.2	Quando scegliere le Asserzioni	94
6.3	Logging [BOZZE DA RIPRENDERE]	95
6.3.1	Architettura del Logging: Facciate e Implementazioni	95
6.3.2	L'interfaccia <code>System.Logger</code>	96
6.3.3	Ottenimento e Utilizzo del Logger	97
7	Generics	99
7.1	Introduzione	99
7.2	Struttura di una Classe Generica	100
7.3	Metodi Generici	101
7.4	Variabili di Tipo Vincolate (Bounded Type Variables)	101
7.5	Funzionamento interno dei Generics	102
7.5.1	Type Erasure (Cancellazione del Tipo)	102
7.5.2	Traduzione delle Espressioni Generiche	103
7.5.3	Type Erasure e Polimorfismo: i Metodi Ponte (Bridge Methods)	103
7.6	Interoperabilità con Codice Legacy	105
7.7	Record Patterns e Generics	106
7.8	Ereditarietà nei Tipi Generici	107
7.9	Wildcard Types (Tipi Jolly)	108
7.9.1	Usi consentiti	108

7.9.2	Wildcard con vincolo inferiore: ? super T	110
7.9.3	Applicazioni in Supertipi Generici	110
7.9.4	Il Principio PECS (Producer Extends, Consumer Super)	112
7.9.5	Unbounded Wildcards	113
7.9.6	Wildcard Capture	113
7.10	Restrizioni e Limitazioni dei Generics	114
7.10.1	Verifiche di Tipo a Runtime	114
7.10.2	Divieto di Creazione di Array Generici	114
7.10.3	Limiti sui Varargs	115
7.10.4	Divieto di Istanziamento di Variabili di Tipo	116
8	Collections	117
8.1	Separazione tra Interfaccia e Implementazione	117
8.2	L'Interfaccia Collection	118
8.2.1	Il Concetto di Operazioni Opzionali	120
8.3	Iteratori	120
8.4	Le Classi Astratte "Ponte"	123
A	Appendice A: Nozioni Accessorie	126
A.1	Il Concetto di Hashing	126
A.2	L'Array Circolare (Circular Array)	127
B	Appendice B: Riferimento Rapido (bozza da sistemare)	128
B.1	Classe String	128
B.2	Classe Math	128
B.3	BigInteger e BigDecimal	128
B.4	Gestione del Tempo (java.time)	128
B.5	Parsing e Conversione	129
C	Bozze e Argomenti in Sospeso	130
C.1	Argomenti in sospeso da riprendere	130
C.2	Il Class Path	130
C.2.1	La "Ricerca Intelligente" del Compilatore	130
C.2.2	I file JAR (Java Archive)	131
C.2.3	L'Algoritmo di Ricerca	131
C.2.4	Uso delle Wildcard	131
C.3	La Classe <code>java.lang.System</code>	131
C.3.1	Caratteristiche Strutturali	131
C.3.2	I Flussi di I/O Standard	132
C.3.3	Gestione del Tempo e Misurazioni	132
C.3.4	Interazione con l'Ambiente Operativo	132
C.3.5	Utility di Sistema e Memoria	133
C.3.6	Integrazione con il Logging Moderno	133
C.4	Snippets vari da inserire	133

Collections

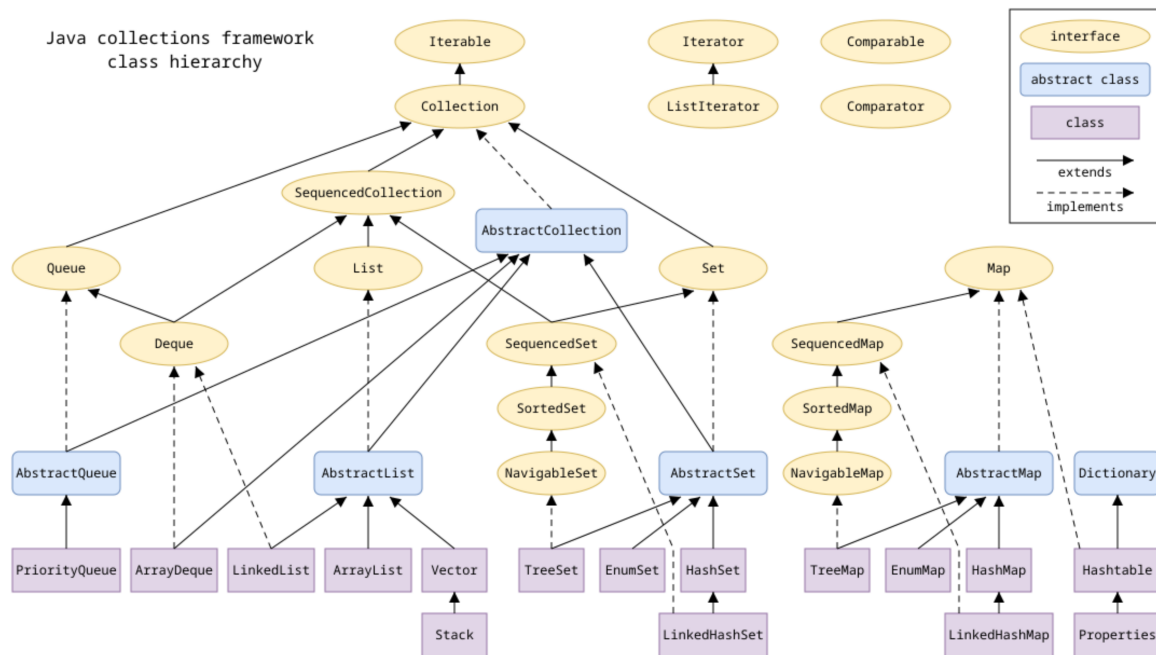


Figura 8.1: Gerarchia delle interfacce del Java Collection Framework

8.1 Separazione tra Interfaccia e Implementazione

Il *Java Collections Framework* adotta un approccio fondamentale della programmazione a oggetti: la netta separazione tra le **interfacce** (che definiscono il comportamento) e le **implementazioni** concrete (che definiscono le strutture dati e le prestazioni). Per illustrare bene questo concetto e i suoi vantaggi, esaminiamo il seguente esempio.

Una *Queue* (coda) specifica un comportamento **FIFO** (*First In, First Out*): aggiunta in coda, rimozione in testa e monitoraggio della dimensione.

Listing 8.1: Interfaccia minimale di una Queue

```
1 public interface Queue<E> {  
2     void add(E e);  
3     E remove();  
4     int size();  
5 }
```

L'interfaccia non dice nulla su come i dati siano memorizzati. Esistono due implementazioni principali nel JDK:

1. **ArrayDeque**: Utilizza un *circular array* (array circolare, vedi in appendice A.2).
2. **LinkedList**: Utilizza una lista concatenata di nodi.

Il vantaggio principale di questa separazione è la flessibilità. Nel codice backend, è considerata una *best practice* utilizzare il tipo interfaccia per dichiarare le variabili, limitando l'uso della classe concreta solo al momento della costruzione.

Listing 8.2: Utilizzo corretto delle interfacce

```

1 // Uso l'interfaccia come tipo della variabile
2 Queue<Customer> expressLane = new ArrayDeque<>(100);
3 expressLane.add(new Customer("Harry"));
4
5 // Se cambio idea, devo cambiare solo la riga del costruttore
6 Queue<Customer> expressLane = new LinkedList<>();

```

✓ Coding to an Interface

Usare il tipo dell'interfaccia (es. `Queue`) invece del tipo concreto (es. `ArrayDeque`) protegge il codice da dipendenze eccessive. Se accidentalmente si utilizzasse un metodo specifico di `ArrayDeque` non presente in `Queue`, il compilatore segnalerebbe l'errore, garantendo che il codice rimanga facilmente intercambiabile.

In Java 6, quando fu introdotta `ArrayDeque`, i programmatori che avevano "programmato per interfacce" poterono migliorare istantaneamente le prestazioni delle loro applicazioni cambiando una singola riga di codice (il costruttore), lasciando intatto tutto il resto della logica aziendale.

Perché scegliere l'una o l'altra? L'interfaccia è agnostica riguardo alle prestazioni, ma le implementazioni no:

- **ArrayDeque**: È generalmente più performante. Ha un impatto minore sul *Garbage Collector* perché non deve creare un oggetto "Nodo" per ogni elemento inserito. Richiede la riallocazione dell'array quando la capacità finisce (operazione computazionalmente costosa), ma se la capacità iniziale è ben calibrata, queste riallocazioni sono rare.
- **LinkedList**: Non ha problemi di capacità finita, ma ogni elemento aggiunto comporta la creazione di un nuovo oggetto in memoria, rendendola meno efficiente in scenari ad alto volume di dati.

8.2 L'Interfaccia Collection

L'interfaccia `Collection<E>` è la radice della gerarchia per le strutture dati che gestiscono gruppi di oggetti (ad eccezione delle *Map*). Essa definisce un set di operazioni standard che garantiscono l'interoperabilità tra diverse implementazioni. Di seguito, esaminiamo i metodi di uso più comune.

Operazioni di Query (Ispezione)

Questi metodi permettono di ottenere informazioni sullo stato della collezione senza modificarla.

- `int size()`: Restituisce il numero di elementi.
- `boolean isEmpty()`: Equivalente a `size() == 0`.
- `boolean contains(Object o)`: Verifica la presenza di un elemento.

Operazioni di Modifica

- `boolean add(E element)`: Restituisce **true** se l'aggiunta ha effettivamente modificato la collezione, **false** altrimenti. Ad esempio, in una *List*, il metodo `add` restituirà quasi sempre **true**; invece, in un *Set* (che non ammette duplicati), se provi ad aggiungere un oggetto già presente, il metodo restituirà **false**, segnalando che l'operazione non ha avuto effetto.
- `boolean remove(Object o)`: Rimuove una singola istanza dell'elemento, se presente. Il valore restituito segue la stessa logica di `add`.
- `void clear()`: Rimuove tutti gli elementi, svuotando la collezione.

Operazioni Bulk (di Massa)

Questi metodi sono fondamentali per manipolare interi set di dati (es. confrontare liste di ID o filtrare risultati). Per `addAll`, `removeAll` e `retainAll` il valore di ritorno indica se la struttura dati è stata mutata, per `containsAll` il `boolean` rappresenta l'esito logico del test di inclusione.

- `boolean addAll(Collection<? extends E> c)`: **Unione**. Aggiunge tutti gli elementi della collezione `c` alla collezione corrente.
- `boolean removeAll(Collection<?> c)`: **Differenza**. Rimuove dalla collezione corrente tutti gli elementi che sono presenti anche in `c`.
- `boolean retainAll(Collection<?> c)`: **Intersezione**. Rimuove dalla collezione corrente tutti gli elementi che **non** sono contenuti in `c`, mantenendo solo i comuni.
- `boolean containsAll(Collection<?> c)`: **Inclusione**. Verifica se la collezione corrente contiene tutti gli elementi della collezione `c`.

Conversione in Array

Nonostante la potenza delle collezioni, a volte è necessario interfacciarsi con API legacy che richiedono array classici.

- `Object[] toArray()`: Restituisce un array di oggetti.
- `<T> T[] toArray(T[] a)`: Versione tipizzata. Se l'array `a` è abbastanza grande, i dati vengono inseriti lì; altrimenti, viene creato un nuovo array dello stesso tipo.

Metodi Default e Approccio Funzionale (Java 8+)

Con l'introduzione delle lambda, l'interfaccia si è arricchita di metodi `default` molto potenti:

- `boolean removeIf(Predicate<? super E> filter)`: Rimuove tutti gli elementi che soddisfano una determinata condizione. È molto più efficiente e pulito di un ciclo manuale con iteratore.
- `Stream<E> stream()`: Il punto d'ingresso per la *Stream API*, essenziale per trasformazioni e filtri complessi in stile dichiarativo. Tratteremo gli *Stream* in un capitolo successivo.



Rischio di Errori a Runtime per Parametri Legacy

Un dettaglio tecnico sottile ma cruciale riguarda il tipo dei parametri di alcuni metodi fondamentali come `contains`, `remove` e le operazioni bulk. Sebbene la

⚠ collezione sia generica (`Collection<E>`), questi metodi accettano parametri di tipo `Object` o `Collection<?>`.

Il *Java Collections Framework* è stato progettato prima dell'introduzione dei tipi generici. Per mantenere la compatibilità con il codice legacy, i parametri di `contains` e `remove` sono rimasti di tipo `Object`. Questo significa che il compilatore non segnalerà errori se si tenta di cercare o rimuovere un oggetto di tipo errato.

Listing 8.3: Esempio di errore di tipo non rilevato

```
1  Collection<Path> paths = ...;
2  // Errore logico: cerco di rimuovere una String da una
   collezione di Path
3  paths.remove("/tmp"); // Compila correttamente, ma non
   avra' alcun effetto
```

8.2.1 Il Concetto di Operazioni Opzionali

Quasi tutti i metodi che abbiamo elencato non sono di default e ciò potrebbe indurre a pensare che *tutte* le collezioni concrete in Java implementino tutti questi metodi. Tuttavia, questo può essere fuorviante: il framework Java gestisce la varietà delle strutture dati (es. collezioni a dimensione fissa o non modificabili) attraverso il concetto di **metodi opzionali**.

Invece di creare decine di sotto-interfacce, i progettisti di Java hanno preferito mantenere l'interfaccia `Collection` unica. Se una specifica implementazione non supporta un metodo (ad esempio il metodo `add` in una collezione a sola lettura), il metodo viene comunque implementato, ma il suo corpo consiste nel lancio di una eccezione:

```
1 public boolean add(E e) {
2     throw new UnsupportedOperationException();
3 }
```

⚠ Attenzione a Runtime

Poiché l'obbligo di implementazione è soddisfatto formalmente, il compilatore non segnalerà errori. L'errore emergerà solo a **runtime**. Questo è un motivo in più per cui è fondamentale conoscere bene la documentazione della classe concreta che si sta istanziando.

8.3 Iteratori

Per scorrere gli elementi di una collezione, il Java Collections Framework non si affida agli indici (che sarebbero inefficienti per strutture come le liste concatenate), ma utilizza l'interfaccia `Iterator<E>`; essa definisce il meccanismo standard per visitare gli elementi di una collezione uno alla volta. A livello astratto, si può immaginare l'iteratore come un cursore posizionato **tra** gli elementi (si veda la figura 8.2).

L'interfaccia `Iterator` definisce quattro metodi principali:

```
1 public interface Iterator<E> {
2     boolean hasNext();
3     E next();
```

```

4  default void remove();
5  default void forEachRemaining(Consumer<? super E> action);
6  }

```

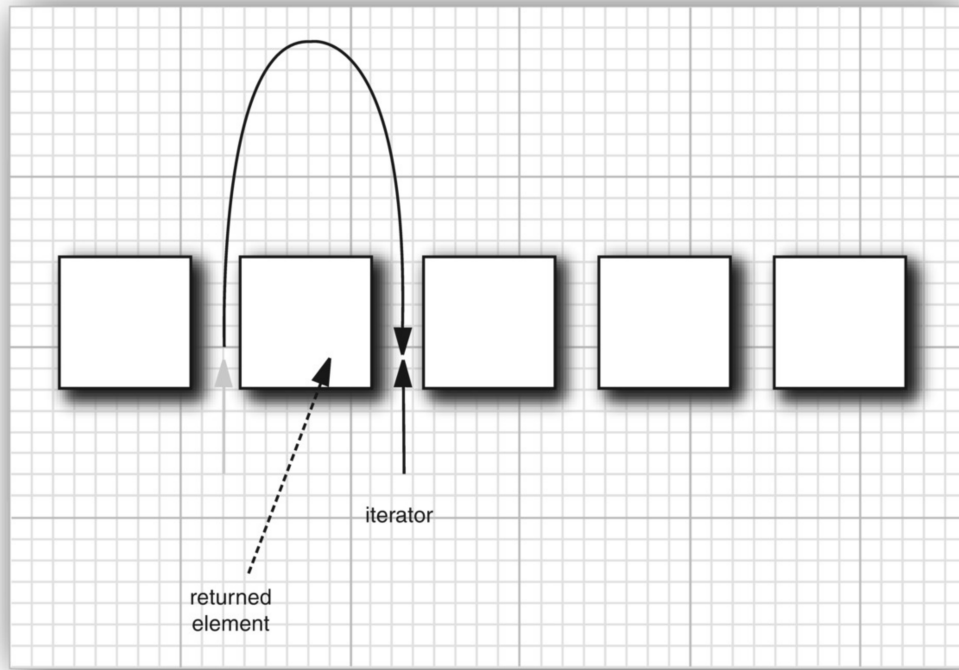


Figura 8.2: l'iteratore come cursore **tra** elementi

- `boolean hasNext()`: Restituisce `true` se ci sono ancora elementi da scorrere.
- E `next()`: Chiamando questo metodo, il cursore "salta" oltre l'elemento successivo e restituisce un riferimento all'elemento che ha appena saltato.
- `void remove()`: Cancella l'elemento restituito dall'ultima chiamata di `next`; esso è soggetto a vincoli: può essere chiamato solo dopo una chiamata a `next` (in concetto è: prima lo devo leggere, poi lo posso cancellare) e solo **una volta** dopo ogni chiamata a `next()` (non si può chiamare due volte di fila senza un `next` in mezzo). Se uno di questi vincoli non viene rispettato, si riceve una `IllegalStateException`.
- `void forEachRemaining(Consumer<? super E> action)`: Esegue un'azione su tutti gli elementi rimanenti.



Da notare che nel codice dell'interfaccia i metodi `remove()` e `forEachRemaining()` sono segnati come default. Questo significa che chi implementa l'interfaccia non è obbligato a scriverne il corpo se non ne ha bisogno.

Nello specifico, `remove()` è un metodo "opzionale": alcune collezioni (come quelle non modificabili) potrebbero lanciare una `UnsupportedOperationException` se provi a usarlo.

Come si vede dalla figura 8.1, `Iterable` rappresenta l'interfaccia padre di `Collection`; ogni classe che la implementa dichiara di poter restituire i propri elementi uno alla volta. Essa contiene un solo metodo astratto e non di default che restituisce un iteratore:

```
1 public interface Iterable<E> {  
2     Iterator<E> iterator();  
3 }
```

Questo metodo viene dunque ereditato da `Collection`; per ispezionare tutti gli elementi di una collezione, si richiede un iteratore e si richiama ripetutamente il metodo `next()`. Se si chiama `next()` quando si è già raggiunta la fine della collezione, l'iteratore solleva una `NoSuchElementException`. Per questo motivo, è fondamentale verificare sempre la presenza di un elemento successivo tramite il metodo `hasNext()`.

Listing 8.4: Utilizzo classico del ciclo while con Iterator

```
1 Collection<String> coll = ...;  
2 Iterator<String> iter = coll.iterator();  
3 while (iter.hasNext()) {  
4     String element = iter.next();  
5     // operazione sull'elemento  
6 }
```

Java permette di scrivere il ciclo precedente in modo molto più conciso grazie al costrutto **for each**. Il compilatore traduce automaticamente questo costrutto in un ciclo con iteratore.

Listing 8.5: Sintassi for each

```
1 for (String element : coll) {  
2     // operazione sull'elemento  
3 }
```

Q Dettagli

```
1 // Scritto dallo sviluppatore:  
2 for (String s : coll) { System.out.println(s); }  
3  
4 // Tradotto dal compilatore:  
5 Iterator<String> it = coll.iterator();  
6 while (it.hasNext()) {  
7     String s = it.next();  
8     System.out.println(s);  
9 }
```

Il ciclo "for each" può essere utilizzato con qualsiasi oggetto che implementi l'interfaccia `Iterable`.



Poiché l'interfaccia `Collection` estende `Iterable`, ogni collezione del framework Java può essere scorsa usando la sintassi *for each*. Prediligi sempre questa sintassi per la leggibilità, a meno che tu non abbia bisogno di rimuovere elementi durante

✓ l'iterazione.

⚠ Effetti Collaterali

Tieni presente che durante un `forEach`, non puoi modificare la struttura della collezione (es. aggiungere o rimuovere elementi), altrimenti Java lancerà una `ConcurrentModificationException`.

Oltre ai cicli tradizionali, il *Collections Framework* mette a disposizione metodi che accettano **lambda expressions** per elaborare gli elementi in modo più conciso e leggibile. Invece di gestire manualmente l'iteratore, possiamo passare un'azione (sotto forma di lambda) direttamente alla collezione o all'iteratore stesso.

Listing 8.6: Esempi di iterazione funzionale

```
1 // Applica l'azione a ogni elemento della collezione
2 myCollection.forEach(element -> System.out.println(element));
3
4 // Applica l'azione agli elementi rimanenti dell'iteratore
5 myIterator.forEachRemaining(element -> System.out.println(element
  ));
```

Entrambi questi metodi utilizzano l'interfaccia funzionale `Consumer<? super E>`. La lambda viene invocata internamente per ogni elemento della collezione senza che lo sviluppatore debba preoccuparsi di chiamare `next()` o `hasNext()`.

Per quanto riguarda l'ordine in cui gli elementi vengono visitati, il comportamento dipende esclusivamente dal tipo di collezione; ad esempio, per un `ArrayList` l'iteratore è **predicibile**: inizia dall'indice 0 e incrementa di uno a ogni passo; per un `HashSet`, l'iteratore visita gli elementi in ordine **essenzialmente casuale** e non si può fare alcuna assunzione sulla sequenza di uscita.

💡 Scelta della Collezione

Se l'ordine dei dati è irrilevante (es. sommare dei totali o contare occorrenze), l'incertezza sull'ordine di visita non è un problema. Se invece l'ordine è parte della logica di business (es. una lista di transazioni cronologiche), è vitale scegliere una collezione che garantisca l'ordinamento come `ArrayList` o `TreeSet`.

8.4 Le Classi Astratte "Ponte"

Per evitare che gli sviluppatori debbano scrivere decine di metodi ogni volta che creano una nuova collezione, il JDK fornisce delle classi astratte chiamate ***Abstract Skeleton Implementations***:

- `AbstractCollection`
- `AbstractList`
- `AbstractSet`

Queste classi implementano già la maggior parte dei metodi (come `containsAll`, `removeAll`, `toString`) basandosi sui metodi fondamentali `iterator` e `size`.

 Tip

Se un giorno dovessi creare una tua collezione personalizzata, non implementare direttamente l'interfaccia `Collection`. Estendi invece `AbstractCollection`: dovrai implementare solo `iterator()` e `size()`, e avrai tutti gli altri metodi (compresi quelli bulk) già pronti e funzionanti.

Per capire come queste classi riescano a implementare metodi complessi senza sapere come sono memorizzati i dati, osserviamo come `AbstractCollection` implementa il metodo `contains(Object o)`.

Listing 8.7: Logica di `AbstractCollection.contains`

```
1 public boolean contains(Object o) {
2     Iterator<E> it = iterator(); // Metodo astratto!
3     if (o == null) {
4         while (it.hasNext())
5             if (it.next() == null) return true;
6     } else {
7         while (it.hasNext())
8             if (o.equals(it.next())) return true;
9     }
10    return false;
11 }
```

Osservazione 8.1. `AbstractCollection` non ha idea se la collezione sia un array o una lista concatenata. Tuttavia, sa che ogni collezione **deve** fornire un `Iterator`. Usando quell'unico metodo (che rimane astratto nella classe skeleton), può fornire un'implementazione funzionante di `contains`, `toArray`, `toString` e molti altri.

Quando una classe concreta estende `AbstractCollection`, il carico di lavoro dello sviluppatore è ridotto, ma non nullo.

- **Metodi Obbligatori:** La classe concreta deve fornire le implementazioni di `iterator()` e `size()`.
- **Collezioni Mutabili:** Se la collezione deve permettere la modifica, è necessario implementare anche il metodo `add()`. In caso contrario, il metodo `add()` ereditato lancerà una `UnsupportedOperationException`.
- **Ottimizzazione:** Sebbene `AbstractCollection` fornisca già le implementazioni di molti metodi concreti, la classe figlia è libera di sovrascriverli se può offrire un'implementazione più efficiente (ad esempio, sfruttando un indice o una tabella hash interna per evitare lo scorrimento lineare).

 Dettagli

L'approccio basato sulle "classi ponte" è oggi considerato parzialmente superato dal punto di vista del design. Sarebbe stato preferibile avere questi metodi direttamente nell'interfaccia `Collection` come metodi `default`, ma questa funzionalità non esisteva quando il framework fu progettato.

Tuttavia, nelle versioni recenti sono stati aggiunti diversi metodi `default`:

- **Stream API:** Metodi per l'elaborazione dei dati (trattati nel Volume II).
- **Conversione:** Metodi `toArray` più flessibili.



- **Rimozione Condizionale:** Il metodo `removeIf`.

Appendice A: Nozioni Accessorie

A.1 Il Concetto di Hashing

L'hashing è uno dei pilastri fondamentali dell'informatica moderna. Non è una tecnologia esclusiva di Java, ma un concetto matematico e logico universale utilizzato per la sintesi e l'identificazione rapida dei dati.

Un **hash** (o *digest*) è il risultato di una funzione che prende un input di lunghezza arbitraria (come un testo, un file o un oggetto complesso) e lo trasforma in una stringa o in un numero di **lunghezza fissa**.

L'analogia più calzante è quella dell'**impronta digitale**:

- Un libro di 1000 pagine è un dato complesso.
- Il suo valore hash è un numero (es. 42) che rappresenta in modo quasi univoco quel libro.
- È molto più facile confrontare due numeri (42 vs 43) che confrontare due libri parola per parola.

Affinché una funzione di hash sia utile, deve rispettare tre criteri:

1. **Deterministicità**: Lo stesso input deve generare *sempre* lo stesso identico output.
2. **Efficienza**: Il calcolo dell'hash deve essere estremamente rapido, indipendentemente dalla dimensione del dato in ingresso.
3. **Effetto Valanga (*Avalanche Effect*)**: Una minima modifica all'input (anche un singolo bit) deve produrre un hash completamente diverso e non correlato al precedente.

Ambiti di Applicazione

L'hashing risolve problemi critici in diversi settori:

- **Sicurezza delle Password**: I sistemi non memorizzano le password in chiaro, ma solo il loro hash. In caso di furto del database, l'attaccante non può risalire alla password originale.
- **Integrità dei Dati**: Attraverso algoritmi come SHA-256 o MD5, è possibile verificare se un file scaricato è integro o se è stato corrotto durante il trasferimento.
- **Strutture Dati (Hash Tables)**: Permette di trovare un elemento in un insieme di milioni di record in tempo costante ($O(1)$), usando l'hash come indice per un "armadietto" (*bucket*) specifico.

Il Problema delle Collisioni

Poiché l'insieme dei possibili input è infinito, mentre l'insieme dei possibili valori hash (numeri interi o stringhe di lunghezza fissa) è finito, esiste la possibilità teorica che due input diversi producano lo stesso hash. Questo evento è chiamato **collisione**. Un buon algoritmo di hash riduce drasticamente la probabilità di collisioni, e i sistemi informatici (come la JVM di Java) sono progettati per gestire correttamente questi rari casi di "omonimia digitale".

A.2 L'Array Circolare (Circular Array)

Nello studio delle collezioni, in particolare quando si analizzano le implementazioni di code (*Queue*) e code a doppia entrata (*Deque*), ci si imbatte nel concetto di **array circolare**.

In un normale array lineare, quando implementiamo una coda, aggiungiamo elementi alla fine e li rimuoviamo dall'inizio.

- Ogni volta che rimuoviamo il primo elemento, dovremmo teoricamente "shiftare" tutti gli altri elementi a sinistra per coprire il buco.
- Questa operazione ha una complessità temporale di $O(n)$, rendendo la struttura inefficiente per grandi moli di dati.

L'array circolare risolve il problema "collegando" virtualmente l'ultima posizione dell'array alla prima. Invece di spostare i dati, spostiamo i **puntatori** (indici) che indicano dove inizia e dove finisce la nostra collezione.

L'implementazione si basa su due indici principali:

1. **head**: Punta alla posizione del primo elemento disponibile (per la rimozione).
2. **tail**: Punta alla posizione successiva all'ultimo elemento (per l'inserimento).

Quando un indice raggiunge la fine dell'array fisico, "salta" alla posizione 0. Matematicamente, questo è gestito tramite l'operatore **modulo**:

$$\text{nuova_posizione} = (\text{posizione_attuale} + 1) \pmod{\text{lunghezza_array}}$$

Questa struttura permette operazioni di inserimento e rimozione in **tempo costante** $O(1)$.

Sebbene circolare, l'array ha comunque una dimensione fisica limitata. Quando l'array è pieno (ovvero quando **head == tail**), Java deve allocare un nuovo array più grande e copiare gli elementi. Tuttavia, questa operazione avviene raramente se la capacità iniziale è ben calibrata.

Appendice B: Riferimento Rapido (bozza da sistemare)

In questa appendice sono raccolti i metodi delle librerie standard di Java 21 necessari per risolvere la maggior parte degli esercizi algoritmici senza consultare la documentazione esterna.

B.1 Classe String

Le stringhe sono immutabili. Ogni metodo restituisce una nuova istanza.

Metodo	Descrizione
<code>length()</code>	Restituisce il numero di caratteri.
<code>charAt(int i)</code>	Carattere alla posizione i .
<code>substring(start, end)</code>	Sottostringa dall'indice <i>start</i> a <i>end-1</i> .
<code>contains(CharSequence)</code>	Verifica se la sequenza è presente.
<code>split(String regex)</code>	Divide la stringa in un array di sottostringhe.
<code>toLowerCase()</code> / <code>toUpperCase()</code>	Cambio di <i>case</i> .
<code>strip()</code> / <code>trim()</code>	Rimuove spazi bianchi (<code>strip</code> è più moderno/Unicode).

B.2 Classe Math

Metodi statici per operazioni matematiche comuni.

Metodo	Esempio / Risultato
<code>Math.min(a, b)</code> / <code>Math.max(a, b)</code>	Restituisce il minore o il maggiore.
<code>Math.abs(x)</code>	Valore assoluto $ x $.
<code>Math.sqrt(x)</code>	Radice quadrata $\sqrt{x}$.
<code>Math.pow(base, esp)</code>	Elevamento a potenza (restituisce <code>double</code>).
<code>Math.round(x)</code>	Arrotondamento all'intero più vicino.

B.3 BigInteger e BigDecimal

Metodi per l'aritmetica ad alta precisione.

Operazione	Sintassi Java
Somma / Sottrazione	<code>a.add(b)</code> / <code>a.subtract(b)</code>
Moltiplicazione / Divisione	<code>a.multiply(b)</code> / <code>a.divide(b, roundingMode)</code>
Confronto	<code>a.compareTo(b)</code> (0 se uguali, 1 se $a > b$, -1 se $a < b$)

B.4 Gestione del Tempo (java.time)

Ricorda: queste classi sono immutabili. Ogni modifica crea un nuovo oggetto.

Metodo	Descrizione / Esempio
<code>LocalDate.now()</code>	Data corrente del sistema.
<code>LocalDate.of(y, m, d)</code>	Crea una data specifica (es. 2026, 4, 2).
<code>date.plusDays(n)</code>	Aggiunge n giorni (restituisce nuovo oggetto).
<code>date.isBefore(other)</code>	Verifica se la data precede un'altra.
<code>ChronoUnit.DAYS.between(a, b)</code>	Calcola i giorni tra due date.

B.5 Parsing e Conversione

Metodi per trasformare stringhe in tipi di dato e viceversa.

Operazione	Sintassi Java
Da String a int	<code>Integer.parseInt("123")</code>
Da String a double	<code>Double.parseDouble("12.5")</code>
Da int/double a String	<code>String.valueOf(n)</code>
Format di output	<code>System.out.printf("%d - %.2f", i, d)</code>

Bozze e Argomenti in Sospeso

C.1 Argomenti in sospeso da riprendere

Cap.4:

- **Factory Methods:** par. 4.4.4.
- **Distruzione di oggetti:** par. 4.6.8.
- **Costruttori canonici nei records:** par. 4.7.2.
- **seconda parte delle lambdas da fare/rivedere:** a partire da par. 6.2.5.
- **logging**
- **seconda parte delle limitazioni sui generics**

C.2 Il Class Path

Il **Class Path** (percorso delle classi) è la collezione di tutte le locazioni (cartelle o archivi) in cui la JVM e il compilatore cercano i file `.class`. È, di fatto, l'elenco dei punti di partenza per la risoluzione dei package.

Il Class Path è una stringa composta da vari percorsi. Il separatore dipende dal sistema operativo:

Sistema	Esempio di Class Path	Separatore
Fedora / UNIX	<code>/home/user/classdir:../libs/archive.jar</code>	Due punti (:)
Windows	<code>C:\classdir;.;C:\libs\archive.jar</code>	Punto e virgola (;)

Osservazione C.1 (Il simbolo del punto). In entrambi i sistemi, il punto (.) rappresenta la **directory corrente**.

È comune confondere il Classpath con la directory di output. Ecco la distinzione corretta:

- **Per il Compilatore (javac):** Il Classpath serve a **risolvere i riferimenti esterni**. Se la tua classe A usa la classe B (che si trova in un JAR o in un'altra cartella), il compilatore deve "leggere" B per verificare che tu stia usando i suoi metodi correttamente.
 - *Nota:* Per decidere **dove mettere** il file compilato, si usa sempre il flag `-d`, non il Classpath.
- **Per la JVM (java):** Il Classpath serve a **caricare il codice in memoria**. La JVM non sa nulla dei file `.java`; lei cerca solo il bytecode (`.class`) seguendo i percorsi indicati nel Classpath per poter avviare l'esecuzione.

C.2.1 La "Ricerca Intelligente" del Compilatore

Horstmann sottolinea che il compilatore è più sofisticato della JVM nella ricerca. Quando incontra una classe, ad esempio `Employee`:

1. Cerca il file `Employee.class` nel Classpath.
2. Se trova anche il file `Employee.java`, controlla le date di modifica.

3. **Auto-recompile:** Se il sorgente (`.java`) è più recente del compilato (`.class`), il compilatore ricompila automaticamente il sorgente per garantire che tu stia usando la versione più aggiornata.

C.2.2 I file JAR (Java Archive)

Oltre alle normali directory, le classi possono essere raggruppate in file **JAR**.

- Un file JAR è essenzialmente un archivio in formato **ZIP** che contiene classi e strutture di sottocartelle compresse.
- È lo standard per la distribuzione di librerie di terze parti.
- Permette di migliorare le prestazioni (un solo file da aprire invece di centinaia) e risparmiare spazio.

C.2.3 L'Algoritmo di Ricerca

Se la JVM deve caricare la classe `com.horstmann.corejava.Employee`, cercherà in ordine:

1. Nelle classi delle **API di Java** (sempre cercate per prime, non serve includerle nel Class Path).
2. In ogni voce del Class Path, aggiungendo il percorso del package alla base. Ad esempio, se il path contiene `/home/user/classdir`, cercherà:
`/home/user/classdir/com/horstmann/corejava/Employee.class`

Attenzione C.1 (La trappola del punto). Per impostazione predefinita, il Class Path è costituito solo dal punto (`.`). Tuttavia, se specifichi un Class Path personalizzato (tramite la variabile d'ambiente `CLASSPATH` o il flag `-cp`) e **dimentichi** di includere il punto (`.`), la JVM non guarderà più nella cartella corrente.

- **Risultato:** Il programma compila (perché `javac` guarda sempre la cartella corrente), ma non parte (perché `java` non la trova).

C.2.4 Uso delle Wildcard

È possibile includere tutti i file JAR di una cartella usando l'asterisco:

```
1 # Su Fedora (bisogna proteggere l'asterisco dalle espansioni
   della shell)
2 java -cp "/home/user/libs/*:." MiaClasse
```

Nota: L'asterisco include tutti i file `.jar`, ma non i singoli file `.class` sparsi nella cartella.

C.3 La Classe `java.lang.System`

La classe `java.lang.System` rappresenta uno dei pilastri fondamentali del Java Development Kit (JDK). Situata nel pacchetto `java.lang`, essa funge da interfaccia primaria tra l'applicazione Java e l'ambiente in cui viene eseguita (Java Virtual Machine e Sistema Operativo).

C.3.1 Caratteristiche Strutturali

Dal punto di vista dell'architettura software, la classe **System** presenta caratteristiche peculiari:

- **Finalità:** È dichiarata come `public final`, il che impedisce qualsiasi tentativo di ereditarietà.

- **Istanziabilità:** Il costruttore è privato, rendendo impossibile creare istanze della classe. Tutte le sue funzionalità sono accessibili esclusivamente tramite metodi e campi `static`.
- **Accessibilità:** Essendo parte di `java.lang`, viene importata automaticamente in ogni file sorgente Java.

C.3.2 I Flussi di I/O Standard

La classe gestisce i tre canali di comunicazione standard definiti a livello di sistema operativo. Questi campi sono istanze di `PrintStream` (per l'output) e `InputStream` (per l'input).

1. `System.out`: Rappresenta lo *standard output*. Viene utilizzato per l'invio di dati alla console o al terminale.
2. `System.err`: Rappresenta lo *standard error*. Sebbene tecnicamente simile a `out`, è destinato specificamente alla diagnostica. Molti IDE e sistemi di logging separano questi due flussi, evidenziando spesso l'errore in rosso.
3. `System.in`: Rappresenta lo *standard input*. Viene solitamente utilizzato in combinazione con la classe `Scanner` per leggere l'input dell'utente dalla tastiera.

Listing C.1: Esempio di utilizzo dei flussi standard

```
1 System.out.println("Messaggio informativo");  
2 System.err.println("Messaggio di errore critico");
```

C.3.3 Gestione del Tempo e Misurazioni

Per il monitoraggio delle prestazioni e la gestione temporale, la classe offre due metodi fondamentali con risoluzioni differenti:

- `currentTimeMillis()`: Restituisce il tempo corrente in millisecondi dall'epoca Unix (1 gennaio 1970). È ideale per calcolare date o timestamp da salvare nel database.
- `nanoTime()`: Restituisce il valore di un timer ad alta risoluzione in nanosecondi. Non è legato all'ora solare, ma è lo strumento standard per misurare quanto tempo impiega un blocco di codice ad essere eseguito (benchmarking).

C.3.4 Interazione con l'Ambiente Operativo

La classe permette di interrogare il sistema operativo e la configurazione della JVM tramite le *System Properties* e le *Environment Variables*.

Proprietà di Sistema

Le proprietà di sistema sono coppie chiave-valore che definiscono il runtime Java.

- `System.getProperty("user.home")`: Restituisce il percorso della cartella utente.
- `System.getProperty("java.version")`: Restituisce la versione di Java in uso.
- `System.lineSeparator()`: Restituisce il separatore di riga specifico dell'OS (`\n` su Unix, `\r\n` su Windows).

Variabili d'Ambiente

Le variabili d'ambiente sono impostate a livello di OS e sono fondamentali nel backend moderno (specialmente con Docker e Kubernetes) per gestire configurazioni sensibili come password o indirizzi di database.

```
1 String dbUrl = System.getenv("DATABASE_URL");
```

C.3.5 Utility di Sistema e Memoria

- **arraycopy()**: Un metodo nativo estremamente performante per copiare dati tra array. A differenza di un ciclo **for**, questo metodo sposta i blocchi di memoria direttamente tramite chiamate a basso livello.
- **gc()**: Suggerisce alla JVM di eseguire il Garbage Collector. È importante notare che è solo un suggerimento e non garantisce l'immediata liberazione della memoria.
- **exit(int status)**: Termina la JVM corrente. Un codice di stato 0 indica una chiusura corretta, mentre un valore diverso da zero indica un fallimento.

C.3.6 Integrazione con il Logging Moderno

Dalla versione 9 di Java, la classe **System** ha assunto un ruolo centrale nella *Platform Logging API* tramite il metodo:

```
1 System.Logger logger = System.getLogger("NomeLogger");
```

Questo permette di astrarre il sistema di logging, delegando a **System** il compito di trovare e utilizzare il miglior backend disponibile (come JUL o Log4j).

Attenzione

Nel contesto dei server e delle applicazioni web, l'uso di **System.exit()** e la manipolazione diretta di **System.out** sono sconsigliati. Si preferisce delegare la chiusura al contenitore dell'applicazione e utilizzare i framework di logging per l'output, garantendo così la stabilità e la tracciabilità del sistema.

C.4 Snippets vari da inserire

— Esiste un paradosso apparente: le interfacce non possono estendere le classi (nemmeno **Object**), eppure possiamo chiamare **.toString()** o **.equals()** su qualsiasi variabile di tipo interfaccia.

Questo accade perché Java aggiunge implicitamente le firme dei metodi pubblici di **Object** a ogni interfaccia.
